

Repo Map — Estrutura do Repositório

Onde encontrar cada coisa no código da Fluxa.

Este documento é o GPS do repositório. Qualquer dev novo deve lê-lo primeiro.

Visão Geral

```
base-x/
├── client/           ← Frontend React (Vite + Tailwind 4)
├── server/           ← Backend Express + tRPC
├── drizzle/          ← Schema do banco + migrações
├── shared/           ← Tipos e constantes compartilhados
├── docs/             ← Documentação (você está aqui)
├── e2e/             ← Testes end-to-end (Playwright)
├── scripts/          ← Scripts utilitários
└── [raiz]            ← Config (vite, vitest, tsconfig, package.json)
```

/client — Frontend

```
client/
├── index.html        ← HTML base (meta tags, fontes, PWA)
├── public/
│   ├── manifest.json ← PWA manifest
│   ├── sw.js         ← Service Worker (offline + push)
│   ├── favicon.svg   ← Ícone
│   ├── icon-192.png / icon-512.png ← PWA icons
│   └── sitemap.xml    ← SEO
└── src/
    ├── App.tsx        ← Rotas + layout principal
    ├── main.tsx       ← Entry point (providers)
    ├── index.css       ← Tema global (CSS vars, Tailwind)
    ├── pages/          ← Páginas (1 arquivo = 1 rota)
    ├── components/     ← Componentes reutilizáveis
    │   ├── ui/         ← shadcn/ui (button, card, dialog...)
    │   └── mission-control/ ← Componentes do Mission Control
    ├── hooks/          ← Custom hooks
    ├── contexts/       ← React contexts (Cart, Theme)
    ├── lib/
    │   └── trpc.ts      ← Cliente tRPC (conexão com backend)
    └── _core/
        └── hooks/useAuth.ts ← Hook de autenticação
```

Páginas por Persona

Persona	Páginas principais
Participante	Menu.tsx , MyOrders.tsx , MyTickets.tsx , TicketPurchase.tsx , Withdrawal.tsx , EventEntry.tsx
Produtor (Admin)	MissionControl.tsx , AdminDashboard.tsx , AdminEvents.tsx , AdminProducts.tsx , AdminOrders.tsx , AdminFinancials.tsx , AdminPostEvent.tsx
Operador (Staff)	StaffConsole.tsx , StaffKDS.tsx , StationKDS.tsx , StaffWithdrawal.tsx
Público (Landing)	Home.tsx , Onboarding.tsx , DemoShowcase.tsx , LiveDemo.tsx , PilotLanding.tsx
Sistema	Login.tsx , Register.tsx , NotFound.tsx , LGPD.tsx , PrivacyPolicy.tsx , TermsOfUse.tsx

/server — Backend

server/	
├── _core/	← Framework (NÃO MEXER sem necessidade)
│ ├── index.ts	← Express app + middlewares + webhooks
│ ├── trpc.ts	← Definição de procedures (public/protected/admin)
│ ├── context.ts	← Context do trpc (user, session)
│ ├── oauth.ts	← Manus OAuth flow
│ ├── cookies.ts	← Configuração de cookies
│ ├── env.ts	← Variáveis de ambiente
│ ├── llm.ts	← Helper para chamar LLM
│ ├── notification.ts	← Push notifications para owner
│ ├── imageGeneration.ts	← Geração de imagens via IA
│ ├── voiceTranscription.ts	← Transcrição de áudio
│ ├── heartbeat.ts	← Jobs periódicos (cron)
│ ├── storageProxy.ts	← Proxy para storage
│ ├── dataApi.ts	← API de dados externos
│ ├── map.ts	← Google Maps proxy
│ ├── sdk.ts	← SDK interno
│ ├── systemRouter.ts	← Router de sistema (notify owner)
│ └── vite.ts	← Integração Vite dev/prod
├── routers/	← Routers trpc (lógica de negócio)
│ ├── orders.ts	← Pedidos + pagamento MP
│ ├── events.ts	← CRUD de eventos
│ ├── products.ts	← Produtos + categorias
│ ├── ingredients.ts	← Insumos + ficha técnica
│ ├── tickets.ts	← Ingressos + check-in
│ ├── fluxaControl.ts	← Mission Control (dashboard ao vivo)
│ ├── intelligence.ts	← Alertas + sugestões IA
│ ├── syntropyPlanner.ts	← Onboarding conversacional
│ ├── eventPlanner.ts	← Planejamento de evento
│ ├── aiInsights.ts	← Insights IA + ações
│ ├── offlineOrders.ts	← Modo offline
│ ├── withdrawal.ts	← Retirada de itens (QR)
│ ├── stationTokens.ts	← Tokens de estação (KDS)
│ ├── stripe.ts	← Pagamento Stripe
│ ├── stripeConnect.ts	← Split via Stripe Connect
│ ├── plans.ts	← Planos de assinatura
│ ├── pilot.ts	← Programa piloto
│ ├── refunds.ts	← Estornos
│ ├── menuTemplates.ts	← Templates de cardápio
│ ├── auth.ts	← Autenticação + MFA
│ ├── seed.ts	← Dados demo
│ └── shared.ts	← staffProcedure (shared)
├── intelligence-engine.ts	← Motor de inteligência (Syntropy runtime)
├── dispatch.ts	← Smart Routing (distribuição de pedidos)
├── scheduled-ia-analysis.ts	← Análise IA periódica (heartbeat)
├── scheduled-invoice-collection.ts	← Cobrança de faturas
├── mercadopago.ts	← SDK Mercado Pago
├── stripe.ts	← SDK Stripe
├── pagarme.ts	← SDK Pagar.me (legado)
├── pushNotification.ts	← Push notifications (VAPID)
├── weather.ts	← Integração clima
├── plans.ts	← Definição de planos
├── sentry.ts	← Monitoramento de erros
├── storage.ts	← S3 helpers
├── db.ts	← Query helpers (Drizzle)
├── ai/	
│ └── safeInvokeLLM.ts	← Wrapper seguro para LLM
└── *.test.ts	← Testes unitários (vitest)

/drizzle — Banco de Dados

```
drizzle/
├── schema.ts           ← TODAS as tabelas (fonte de verdade)
├── relations.ts        ← Relações entre tabelas
├── 0000_*.sql ... 0054_*.sql ← Migrações SQL (em ordem)
└── meta/
    ├── _journal.json   ← Histórico de migrações
    └── *.snapshot.json ← Snapshots do schema
```

/shared — Compartilhado

```
shared/
├── const.ts           ← Constantes (status, enums)
├── types.ts           ← Tipos TypeScript compartilhados
└── _core/
    └── errors.ts       ← Tipos de erro padrão
```

Raiz — Configuração

Arquivo	Função
package.json	Dependências + scripts
vite.config.ts	Build config (client + server)
vitest.config.ts	Config de testes
tsconfig.json	TypeScript config
AGENTS.md	Constituição para IAs/devs
todo.md	Backlog de features/bugs

Arquivos Legados (raiz)

Estes arquivos na raiz são documentação/QA de sprints anteriores. Não fazem parte do runtime:

```
AUDIT-BP-v16-ALIGNMENT.md, AUDITORIA-FLUXA.md, BRAND-ARCHITECTURE.md,
FEATURES.md, FLUXA-IDENTIDADE.md, FLUXA-MAPA-DE-FLUXOS.md,
QA-*.md, ROADMAP_EVENTO_REAL.md, SPEC-MISSION-CONTROL.md, etc.
```

Convenções

Convenção	Regra
Nomes de arquivo	<code>camelCase.ts</code> para código, <code>UPPER_CASE.md</code> para docs
Páginas	1 arquivo = 1 rota (nome = funcionalidade)
Routers	1 arquivo = 1 domínio (orders, events, products...)
Testes	<code>*.test.ts</code> ao lado do arquivo testado
Migrações	Nunca editar manualmente — usar <code>pnpm drizzle-kit generate</code>
Imagens	Nunca no repo — usar <code>manus-upload-file --webdev</code>

Feature → File Map

Precisa mexer em algo? Encontre aqui qual arquivo abrir.

Fluxo do Participante

Funcionalidade	Backend	Frontend	Schema
Ver cardápio	<code>routers/products.ts</code>	<code>pages/Menu.tsx</code>	<code>products</code> , <code>categories</code>
Fazer pedido	<code>routers/orders.ts</code>	<code>pages/Menu.tsx</code> (CartContext)	<code>orders</code> , <code>orderItems</code>
Pagar com Pix (MP)	<code>routers/orders.ts</code> , <code>mercadopago.ts</code>	<code>pages/Menu.tsx</code>	<code>orders</code>
Pagar com cartão (MP)	<code>routers/stripe.ts</code> (payMpCard)	<code>components/MercadoPagoCardForm.tsx</code>	<code>orders</code>
Pagar com cartão (Stripe)	<code>routers/stripe.ts</code>	<code>components/StripePaymentForm.tsx</code>	<code>orders</code>
Apple/Google Pay	<code>routers/stripe.ts</code>	<code>components/ExpressCheckout.tsx</code>	<code>orders</code>
Acompanhar pedido	<code>routers/orders.ts</code> (queuePosition)	<code>pages/MyOrders.tsx</code>	<code>orders</code>
Retirar item (QR)	<code>routers/withdrawal.ts</code>	<code>pages/Withdrawal.tsx</code>	<code>withdrawalTokens</code>
Comprar ingresso	<code>routers/tickets.ts</code>	<code>pages/TicketPurchase.tsx</code>	<code>ticketTypes</code> , <code>ticketPurchases</code>
Meus ingressos	<code>routers/tickets.ts</code>	<code>pages/MyTickets.tsx</code>	<code>ticketPurchases</code>
Push notifications	<code>pushNotification.ts</code>	<code>hooks/usePushNotifications.ts</code>	<code>pushSubscriptions</code>

Fluxo do Produtor (Admin)

Funcionalidade	Backend	Frontend	Schema
Mission Control (ao vivo)	<code>routers/fluxaControl.ts</code>	<code>pages/MissionControl.tsx</code>	múltiplas
Criar evento (Syntropy)	<code>routers/syntropyPlanner.ts</code>	<code>pages/Onboarding.tsx</code> , <code>SyntropyEventCreator.tsx</code>	<code>events</code>
Planejar evento	<code>routers/eventPlanner.ts</code>	<code>pages/AdminEventPlanner.tsx</code>	<code>events</code>
Gerenciar eventos	<code>routers/events.ts</code>	<code>pages/AdminEvents.tsx</code>	<code>events</code>
Gerenciar produtos	<code>routers/products.ts</code>	<code>pages/AdminProducts.tsx</code>	<code>products</code> , <code>categories</code>
Gerenciar insumos	<code>routers/ingredients.ts</code>	<code>pages/AdminIngredients.tsx</code>	<code>ingredients</code> , <code>productIngredients</code>
Monitorar estoque	<code>routers/fluxaControl.ts</code>	<code>pages/AdminStockMonitor.tsx</code>	<code>stockMovements</code> , <code>ingredients</code>
Reconciliar estoque	<code>routers/ingredients.ts</code>	<code>pages/AdminStockReconciliation.tsx</code>	<code>ingredientMovements</code>
Ver pedidos	<code>routers/orders.ts</code>	<code>pages/AdminOrders.tsx</code>	<code>orders</code> , <code>orderItems</code>
Financeiro	<code>routers/fluxaControl.ts</code>	<code>pages/AdminFinancials.tsx</code>	<code>orders</code> , <code>splitTransactions</code>
Recebimentos	<code>routers/stripeConnect.ts</code>	<code>pages/AdminRecebimentos.tsx</code>	<code>splitConfigs</code>
Split (repasse)	<code>routers/stripeConnect.ts</code>	<code>pages/AdminSplit.tsx</code>	<code>splitConfigs</code> , <code>splitTransactions</code>
Promoções	<code>routers/fluxaControl.ts</code>	<code>pages/AdminPromos.tsx</code>	<code>promos</code>
Ingressos	<code>routers/tickets.ts</code>	<code>pages/AdminTickets.tsx</code>	<code>ticketTypes</code>
Check-in	<code>routers/tickets.ts</code>	<code>pages/AdminCheckIn.tsx</code>	<code>ticketPurchases</code> , <code>eventEntries</code>
Open Bar	<code>routers/events.ts</code>	<code>pages/AdminOpenBar.tsx</code>	<code>openBarTracking</code>
CRM	<code>routers/fluxaControl.ts</code>	<code>pages/AdminCRM.tsx</code>	<code>participantProfiles</code>
Contratos	<code>routers/events.ts</code>	<code>pages/AdminContracts.tsx</code> , <code>ContractSign.tsx</code>	<code>contracts</code>
Leads	<code>routers/events.ts</code>	<code>pages/AdminLeads.tsx</code>	<code>leads</code>
Analytics	<code>routers/fluxaControl.ts</code>	<code>pages/AdminAnalytics.tsx</code>	múltiplas
Relatório pós-evento	<code>intelligence-engine.ts</code>	<code>pages/AdminPostEvent.tsx</code>	<code>aiInsights</code> , <code>iaActions</code>
Syntropy Vision	<code>intelligence-engine.ts</code>	<code>pages/AdminSyntropyVision.tsx</code>	<code>aiInsights</code>

Funcionalidade	Backend	Frontend	Schema
Inteligência (alertas)	<code>routers/intelligence.ts</code>	<code>pages/AdminIntelligence.tsx</code>	<code>aiInsights</code> , <code>iaActions</code>
AI Manager	<code>routers/aiInsights.ts</code>	<code>pages/AdminAIManager.tsx</code>	<code>aiInsights</code> , <code>iaActions</code>
Pontos de venda	<code>routers/events.ts</code>	<code>pages/AdminSalePoints.tsx</code>	<code>salePoints</code>
QR Codes	<code>routers/events.ts</code>	<code>pages/AdminQRCodes.tsx</code>	<code>tablesQr</code>
Segurança	<code>routers/auth.ts</code>	<code>pages/AdminSecurityDashboard.tsx</code>	<code>securityAlerts</code> , <code>auditLogs</code>
Planos	<code>routers/plans.ts</code>	<code>pages/AdminPlano.tsx</code>	—
Faturas	<code>scheduled-invoice-collection.ts</code>	<code>pages/AdminFaturas.tsx</code>	<code>eventInvoices</code>
Estornos	<code>routers/refunds.ts</code>	<code>pages/AdminRefunds.tsx</code>	<code>refundRequests</code> , <code>paymentRefunds</code>
Programa piloto	<code>routers/pilot.ts</code>	<code>pages/AdminPilotInvites.tsx</code>	<code>pilotInvites</code> , <code>pilotFeedback</code>

Fluxo do Operador (Staff)

Funcionalidade	Backend	Frontend	Schema
Console de pedidos	<code>routers/orders.ts</code>	<code>pages/StaffConsole.tsx</code>	<code>orders</code>
KDS (Kitchen Display)	<code>routers/orders.ts</code> , <code>stationTokens.ts</code>	<code>pages/StaffKDS.tsx</code> , <code>StationKDS.tsx</code>	<code>orders</code> , <code>stationTokens</code>
Retirada (staff)	<code>routers/withdrawal.ts</code>	<code>pages/StaffWithdrawal.tsx</code>	<code>withdrawalTokens</code>
Pedidos offline	<code>routers/offlineOrders.ts</code>	hooks: <code>useOfflineStaff.ts</code>	<code>offlineOrders</code>
Push (staff)	<code>pushNotification.ts</code>	—	<code>staffPushSubscriptions</code>

Inteligência (Syntropy)

Funcionalidade	Backend	Frontend
Motor de inteligência	intelligence-engine.ts	—
Smart Routing	dispatch.ts	—
Análise periódica	scheduled-ia-analysis.ts	—
Previsão climática	weather.ts	—
Onboarding conversacional	routers/syntropyPlanner.ts	pages/Onboarding.tsx
Relatório pós-evento (Vision)	intelligence-engine.ts	pages/AdminPostEvent.tsx
Alertas + sugestões	routers/intelligence.ts , aiInsights.ts	pages/AdminIntelligence.tsx
CRV (Customer Revenue Value)	routers/fluxaControl.ts	pages/AdminCRV.tsx

Infraestrutura

Funcionalidade	Arquivo
Webhooks MP	server/_core/index.ts (rota /api/webhooks/mercadopago)
Webhooks Stripe	server/_core/index.ts (rota /api/webhooks/stripe)
OAuth	server/_core/oauth.ts
Push VAPID	server/pushNotification.ts
Service Worker	client/public/sw.js
Jobs periódicos	server/_core/heartbeat.ts
Sentry	server/sentry.ts
Storage (S3)	server/storage.ts

Backend Architecture

Todos os routers, suas responsabilidades e níveis de criticidade.

Stack

- **Runtime:** Node.js 22 (single process)
- **Framework:** Express 4 + tRPC 11
- **ORM:** Drizzle (MySQL/TiDB)
- **Auth:** Manus OAuth + JWT cookies
- **Serialização:** Superjson (Date, BigInt preservados)

Níveis de Acesso

Procedure	Quem pode usar	Onde definido
publicProcedure	Qualquer pessoa (sem login)	server/_core/trpc.ts
protectedProcedure	Usuário logado	server/_core/trpc.ts
adminProcedure	Usuário com role=admin	server/_core/trpc.ts
staffProcedure	Staff autenticado por token	server/routers/shared.ts

Routers — Visão Geral

CRÍTICOS (nunca alterar sem teste completo)

Router	Responsabilidade	Risco se quebrar
orders.ts	Criar pedido, pagamento Pix/MP, webhook, status	Perda de receita
stripe.ts	Pagamento cartão, MP Card, Express Checkout	Perda de receita
fluxaControl.ts	Dashboard ao vivo, métricas, estoque, CRM	Produtor cego
intelligence-engine.ts	Motor Syntropy (alertas, sugestões, Vision)	IA para
dispatch.ts	Smart Routing (distribuição de pedidos)	Pedidos perdidos
stationTokens.ts	Autenticação de estações KDS	Bar não recebe pedidos
withdrawal.ts	Retirada de itens via QR	Participante não retira

IMPORTANTES (testar antes de deploy)

Router	Responsabilidade
events.ts	CRUD de eventos, pontos de venda, contratos, QR codes, leads
products.ts	Produtos, categorias, imagens, estoque
ingredients.ts	Insumos, ficha técnica, movimentações, reconciliação
tickets.ts	Tipos de ingresso, compra, confirmação, check-in
stripeConnect.ts	Split de pagamentos, onboarding Stripe Connect
offlineOrders.ts	Pedidos offline (sync, pagamento posterior)
refunds.ts	Estornos (request, approve, reject)
auth.ts	Login, MFA, sessões, segurança

AUXILIARES (menor risco)

Router	Responsabilidade
<code>syntropyPlanner.ts</code>	Chat conversacional para criar evento
<code>eventPlanner.ts</code>	Planejamento detalhado de evento
<code>aiInsights.ts</code>	CRUD de insights IA + ações
<code>intelligence.ts</code>	Listagem de alertas + sugestões
<code>menuTemplates.ts</code>	Templates de cardápio reutilizáveis
<code>plans.ts</code>	Planos de assinatura (Starter, Pro, Enterprise)
<code>pilot.ts</code>	Programa piloto (convites, feedback)
<code>seed.ts</code>	Criar dados demo

Serviços Standalone (fora dos routers)

Arquivo	Função	Trigger
<code>intelligence-engine.ts</code>	Motor Syntropy completo	Chamado por heartbeat + routers
<code>dispatch.ts</code>	Smart Routing de pedidos	Chamado por orders.ts ao criar pedido
<code>scheduled-ia-analysis.ts</code>	Análise IA periódica	Heartbeat (cron)
<code>scheduled-invoice-collection.ts</code>	Cobrança de faturas	Heartbeat (cron)
<code>mercadopago.ts</code>	SDK MP (criar pagamento, verificar)	Chamado por orders.ts
<code>stripe.ts</code> (server/)	SDK Stripe (payment intent)	Chamado por routers/stripe.ts
<code>pushNotification.ts</code>	Enviar push (VAPID)	Chamado por orders, intelligence
<code>weather.ts</code>	Buscar previsão do tempo	Chamado por intelligence-engine
<code>plans.ts</code> (server/)	Definição de planos e limites	Importado por routers/plans.ts

Webhooks

Endpoint	Origem	O que faz
<code>POST /api/webhooks/mercadopago</code>	Mercado Pago	Confirma pagamento → atualiza order → push → estoque
<code>POST /api/webhooks/stripe</code>	Stripe	Confirma payment_intent → atualiza order

Localização: `server/_core/index.ts` (registrados antes do tRPC middleware)

Heartbeat (Jobs Periódicos)

Job	Frequência	Arquivo
Análise IA	A cada 5 min (durante evento)	<code>scheduled-ia-analysis.ts</code>
Cobrança de faturas	Diário	<code>scheduled-invoice-collection.ts</code>

Config: `server/_core/heartbeat.ts`

Fluxo de um Request



1. Request chega no Express (`server/_core/index.ts`)
2. Middleware de segurança (HSTS, CSP, rate limit)
3. Se webhook → rota Express direta (não passa pelo tRPC)
4. Se `/api/trpc/*` → tRPC handler
5. Context é criado (`context.ts`) → extrai user do cookie
6. Procedure verifica auth level
7. Input validado via Zod
8. Lógica de negócio executa
9. Resposta serializada via Superjson

Testes

Tipo	Localização	Comando
Unitários	<code>server/*.test.ts</code>	<code>pnpm test</code>
E2E	<code>e2e/smoke.spec.ts</code>	<code>pnpm playwright test</code>

Total de arquivos de teste: 35+

Regra: Todo router crítico tem pelo menos 1 arquivo de teste.

Frontend Architecture

Páginas, componentes, layouts e fluxos de cada persona.

Stack

- **Framework:** React 19 (SPA)
- **Build:** Vite 6
- **Styling:** Tailwind CSS 4 + shadcn/ui

- **Routing:** Wouter (leve, sem React Router)
- **Data:** tRPC client (React Query under the hood)
- **State:** React Context (Cart, Theme) + hooks
- **PWA:** Service Worker + manifest + push

Layouts

Layout	Usado por	Descrição
DashboardLayout.tsx	Todas as páginas /admin/* e /mission-control	Sidebar + header + auth check
Sem layout (full page)	Home.tsx , Menu.tsx , Onboarding.tsx , Login.tsx	Páginas públicas

Páginas por Fluxo

Participante (público, sem login obrigatório)

```
Home.tsx → Onboarding.tsx → (cria evento)
  → Menu.tsx → (faz pedido) → MyOrders.tsx
  → TicketPurchase.tsx → MyTickets.tsx
  → EventEntry.tsx (entrada no evento)
  → Withdrawal.tsx (retirada de itens)
```

Detalhe do Menu.tsx:

- É a página mais complexa do frontend
- Contém: cardápio, carrinho, checkout (Pix + Cartão MP + Stripe)
- Usa CartContext.tsx para estado do carrinho
- Integra: StripePaymentForm , MercadoPagoCardForm , ExpressCheckout

Produtor (logado, role=admin)

```
MissionControl.tsx (dashboard ao vivo)
├── AdminEvents.tsx (listar/criar eventos)
├── AdminProducts.tsx (cardápio)
├── AdminIngredients.tsx (insumos)
├── AdminOrders.tsx (pedidos)
├── AdminFinancials.tsx (financeiro)
├── AdminStockMonitor.tsx (estoque ao vivo)
├── AdminIntelligence.tsx (alertas Syntropy)
├── AdminPostEvent.tsx (relatório pós-evento / Vision)
├── AdminCRM.tsx (participantes)
├── AdminPromos.tsx (promoções)
├── AdminTickets.tsx (ingressos)
├── AdminCheckIn.tsx (check-in)
├── AdminContracts.tsx (contratos)
├── AdminSplit.tsx (repasse)
├── AdminRecebimentos.tsx (recebimentos)
└── ... (30+ páginas admin)
```

Operador (Staff)

StaffConsole.[tsx](#) (lista de pedidos para preparar)
StaffKDS.[tsx](#) (Kitchen Display System – tela do bar)
StationKDS.[tsx](#) (KDS por estação, autenticado por token)
StaffWithdrawal.[tsx](#) (confirmar retirada de itens)

Landing / Marketing

Home.[tsx](#) (landing page principal)
Onboarding.[tsx](#) (Syntropy conversacional)
DemoShowcase.[tsx](#) (demo interativo)
LiveDemo.[tsx](#) (Mission Control em modo demo)
PilotLanding.[tsx](#) (página do programa piloto)
HowItWorks.[tsx](#) (como funciona)

Componentes Reutilizáveis

Componente	Função	Usado em
<code>DashboardLayout.tsx</code>	Layout admin com sidebar	Todas páginas admin
<code>AIChatBox.tsx</code>	Chat com Syntropy	Onboarding, EventCreator
<code>StripePaymentForm.tsx</code>	Formulário de cartão Stripe	Menu (checkout)
<code>MercadoPagoCardForm.tsx</code>	Formulário de cartão MP	Menu (checkout)
<code>ExpressCheckout.tsx</code>	Apple/Google Pay	Menu (checkout)
<code>AnimatedQR.tsx</code>	QR Code animado	Withdrawal, Tickets
<code>FluxaQR.tsx</code>	QR Code estático	QR Codes admin
<code>QrScanner.tsx</code>	Scanner de QR	StaffWithdrawal, CheckIn
<code>OfflineBanner.tsx</code>	Banner de modo offline	Global
<code>OfflineIndicator.tsx</code>	Indicador offline	Staff pages
<code>SyntropyLiveFeed.tsx</code>	Feed ao vivo da Syntropy	MissionControl
<code>Full7BlocksLayer.tsx</code>	Relatório 7 blocos	PostEvent
<code>ComingSoonPanel.tsx</code>	Placeholder de feature	Várias
<code>ErrorBoundary.tsx</code>	Captura erros React	App.tsx
<code>DateRangePicker.tsx</code>	Seletor de período	Analytics
<code>ManusDialog.tsx</code>	Dialog padrão	Várias
<code>FluxaLogo.tsx</code>	Logo SVG	Header, Login
<code>Redirect.tsx</code>	Redirect helper	App.tsx
<code>mission-control/</code>	Componentes do MC	MissionControl.tsx
<code>ui/</code>	shadcn/ui (button, card...)	Todas

Hooks Customizados

Hook	Função
<code>useAuth.ts</code>	Estado de autenticação (user, login URL)
<code>useOffline.ts</code>	Detecta se está offline
<code>useOfflineStaff.ts</code>	Modo offline para staff (queue local)
<code>useOfflineSync.ts</code>	Sincronização offline → online
<code>useOfflineWithdrawals.ts</code>	Retiradas offline
<code>usePushNotifications.ts</code>	Registrar/receber push
<code>useDocumentTitle.ts</code>	Título dinâmico da página
<code>useMobile.tsx</code>	Detecta mobile
<code>useLongPress.ts</code>	Gesto de long press
<code>useComposition.ts</code>	Composição de componentes
<code>usePersistFn.ts</code>	Função persistente (ref)

Contexts

Context	Função	Usado em
<code>CartContext.tsx</code>	Estado do carrinho (itens, total, add/remove)	Menu.tsx
<code>ThemeContext.tsx</code>	Tema (dark/light)	Global

PWA

Arquivo	Função
<code>client/public/manifest.json</code>	Configuração PWA (nome, ícones, cores)
<code>client/public/sw.js</code>	Service Worker (cache, push, offline)
<code>hooks/usePushNotifications.ts</code>	Registro de push no frontend

Padrões de Código

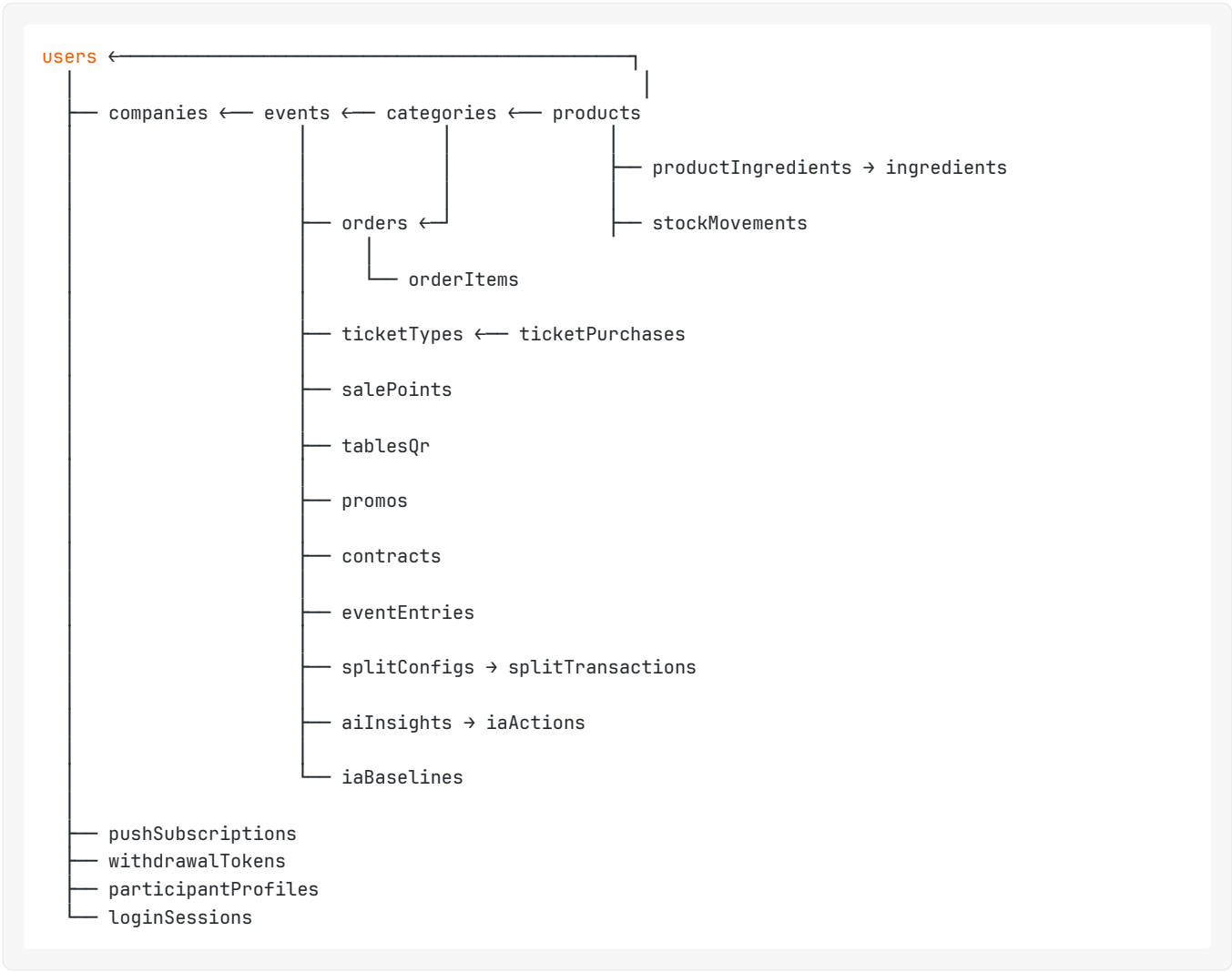
- Data fetching:** Sempre via `trpc.*.useQuery()` ou `trpc.*.useMutation()`
- Loading states:** Skeleton ou spinner por componente (nunca bloqueia página inteira)
- Erro:** `ErrorBoundary` global + toast por operação
- Navegação:** `useLocation()` do Wouter (não React Router)
- Formulários:** Controlled components + Zod validation no backend
- Imagens:** Sempre URL externa (S3) — nunca local

Data Model

Tabelas do banco, o que guardam e como se relacionam.

Schema completo: `drizzle/schema.ts` | Relações: `drizzle/relations.ts`

Diagrama de Dependências (simplificado)



Tabelas Principais

Core

Tabela	O que guarda	Chave estrangeira
<code>users</code>	Todos os usuários (participantes, produtores, staff)	—
<code>companies</code>	Empresas produtoras	<code>userId</code> (owner)
<code>events</code>	Eventos criados	<code>userId</code> , <code>companyId</code>

Cardápio

Tabela	O que guarda	Chave estrangeira
categories	Categorias de produtos (Drinks, Comida...)	eventId
products	Itens do cardápio	categoryId , eventId
menuTemplates	Templates de cardápio reutilizáveis	userId
menuTemplateCategories	Categorias do template	templateId
menuTemplateItems	Itens do template	categoryId

Pedidos e Pagamentos

Tabela	O que guarda	Chave estrangeira
orders	Pedidos (status, pagamento, total)	eventId , userId
orderItems	Itens de cada pedido	orderId , productId
offlineOrders	Pedidos feitos offline (sync posterior)	eventId , userId
refundRequests	Solicitações de estorno	orderId , eventId
paymentRefunds	Estornos processados	orderId

Ingressos

Tabela	O que guarda	Chave estrangeira
ticketTypes	Tipos de ingresso (VIP, Pista...)	eventId
ticketPurchases	Ingressos comprados	ticketTypeId , userId
eventEntries	Check-ins realizados	eventId , userId

Estoque e Insumos

Tabela	O que guarda	Chave estrangeira
stockMovements	Movimentações de estoque de produtos	productId , eventId
ingredients	Insumos (limão, gelo, vodka...)	eventId
productIngredients	Ficha técnica (produto → insumos)	productId , ingredientId
ingredientMovements	Movimentações de insumos	ingredientId
ingredientTemplates	Templates de insumos	userId
ingredientTemplateItems	Itens do template de insumos	templateId

Financeiro

Tabela	O que guarda	Chave estrangeira
splitConfigs	Config de split (% produtor vs plataforma)	eventId
splitTransactions	Transações de split executadas	splitConfigId , orderId
platformRevenue	Receita da plataforma	eventId
eventInvoices	Faturas de eventos	eventId
platformFixedCosts	Custos fixos da plataforma	—
platformTaxConfig	Configuração de impostos	—

Inteligência (Syntropy)

Tabela	O que guarda	Chave estrangeira
iaBaselines	Baselines de métricas (para detectar anomalias)	eventId
aiInsights	Insights gerados pela IA	eventId
iaActions	Ações executadas pela IA	insightId
iaAnalysisLog	Log de análises periódicas	eventId
platformIntelligence	Inteligência cross-evento	—
agentLogs	Logs do agente IA	eventId

Participantes e CRM

Tabela	O que guarda	Chave estrangeira
participantProfiles	Perfil de consumo do participante	userId , eventId
participantFeedback	Feedback pós-evento	eventId , userId
leads	Leads capturados	eventId

Operacional

Tabela	O que guarda	Chave estrangeira
salePoints	Pontos de venda (Bar 1, Bar 2...)	eventId
tablesQr	QR codes de mesas	eventId
rounds	Rodadas (grupo de pedidos)	eventId
roundParticipants	Participantes de rodadas	roundId , userId
withdrawalTokens	Tokens de retirada (QR)	orderId , userId
stationTokens	Tokens de estação KDS	eventId , salePointId
openBarTracking	Tracking de open bar	eventId , userId

Promoções e Auto-consumo

Tabela	O que guarda	Chave estrangeira
<code>promos</code>	Promoções ativas	<code>eventId</code>
<code>autoConsumptionRules</code>	Regras de consumo automático	<code>eventId</code>
<code>autoConsumptionSessions</code>	Sessões de auto-consumo	<code>ruleId</code> , <code>userId</code>

Segurança

Tabela	O que guarda	Chave estrangeira
<code>securityAlerts</code>	Alertas de segurança	<code>eventId</code>
<code>auditLogs</code>	Log de auditoria	<code>userId</code>
<code>mfaSecrets</code>	Secrets MFA (2FA)	<code>userId</code>
<code>loginSessions</code>	Sessões ativas	<code>userId</code>
<code>bankChangeConfirmations</code>	Confirmações de troca bancária	<code>userId</code>

Plataforma

Tabela	O que guarda	Chave estrangeira
<code>pushSubscriptions</code>	Subscriptions de push (participantes)	<code>userId</code>
<code>staffPushSubscriptions</code>	Subscriptions de push (staff)	<code>userId</code>
<code>wallets</code>	Carteiras (legado)	<code>userId</code>
<code>topups</code>	Recargas (legado)	<code>walletId</code>
<code>transactions</code>	Transações (legado)	<code>walletId</code>
<code>pilotInvites</code>	Convites do programa piloto	—
<code>pilotFeedback</code>	Feedback do piloto	<code>inviteId</code>
<code>pilotFollowups</code>	Follow-ups do piloto	<code>inviteId</code>
<code>userFeedback</code>	Feedback geral	<code>userId</code>
<code>contracts</code>	Contratos digitais	<code>eventId</code>
<code>companyMemberRoles</code>	Roles de membros da empresa	<code>companyId</code> , <code>userId</code>

Notas Importantes

- 1. **Tabelas legado:** `wallets` , `topups` , `transactions` existem no schema mas não são mais usadas (modelo antigo de recarga). Não deletar — podem ter dados históricos.
- 2. **Migrações:** 55 migrações aplicadas (0000 a 0054). Nunca editar SQL gerado. Sempre usar `pnpm drizzle-kit generate` .
- 3. **Banco:** MySQL (TiDB) — serverless, auto-scale.
- 4. **IDs:** Todos `int` auto-increment (não UUID).

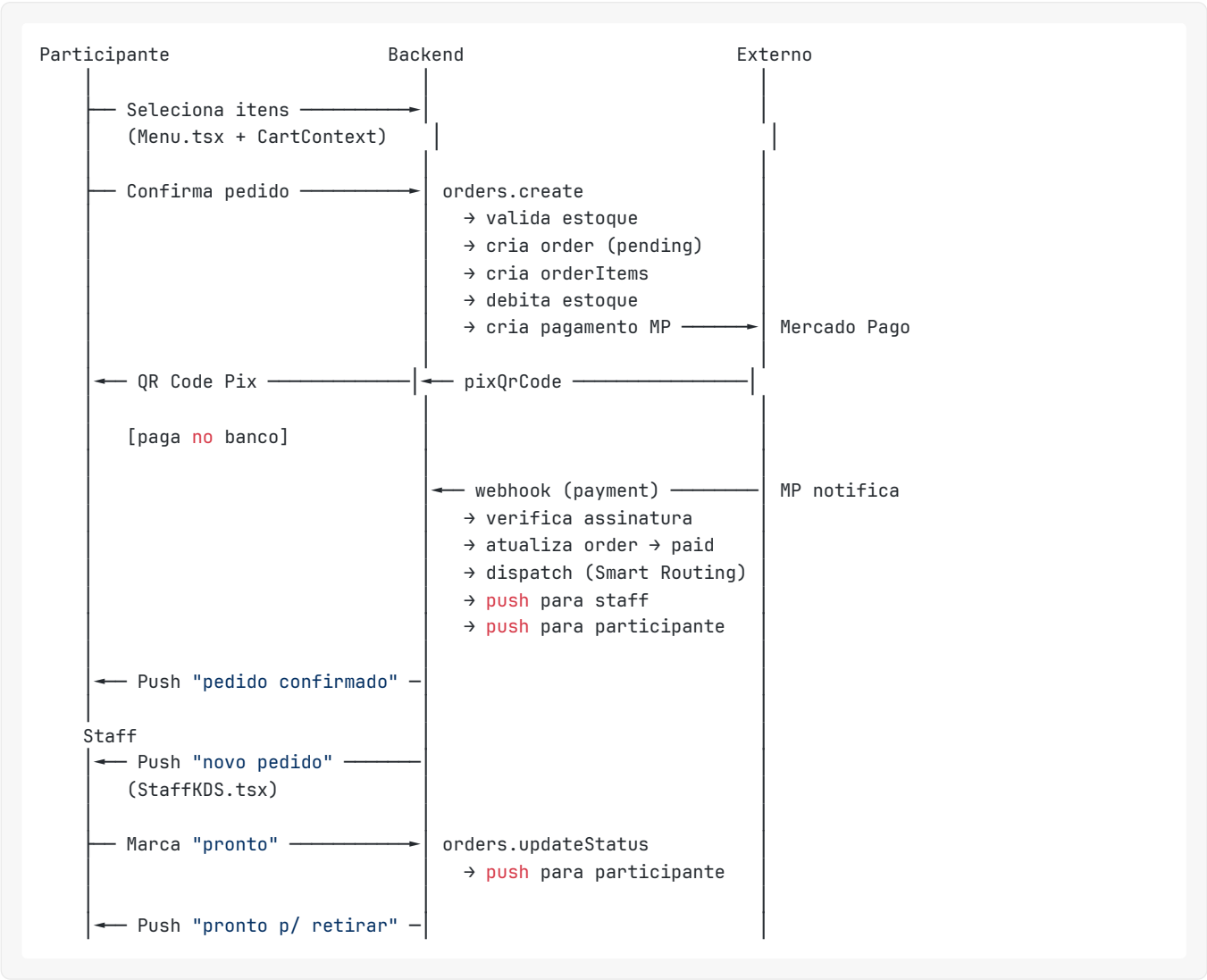
5. **Timestamps:** Armazenados como `bigint` (Unix ms UTC). Frontend converte para local.

Critical Flows

Fluxos que nenhum dev pode quebrar. Se quebrar, o evento para.
Antes de alterar qualquer arquivo listado aqui, rode os testes correspondentes.

1. Pedido → Pagamento → Webhook → Estoque → KDS → Push

O fluxo mais crítico da Fluxa. Se quebrar, ninguém compra nada.



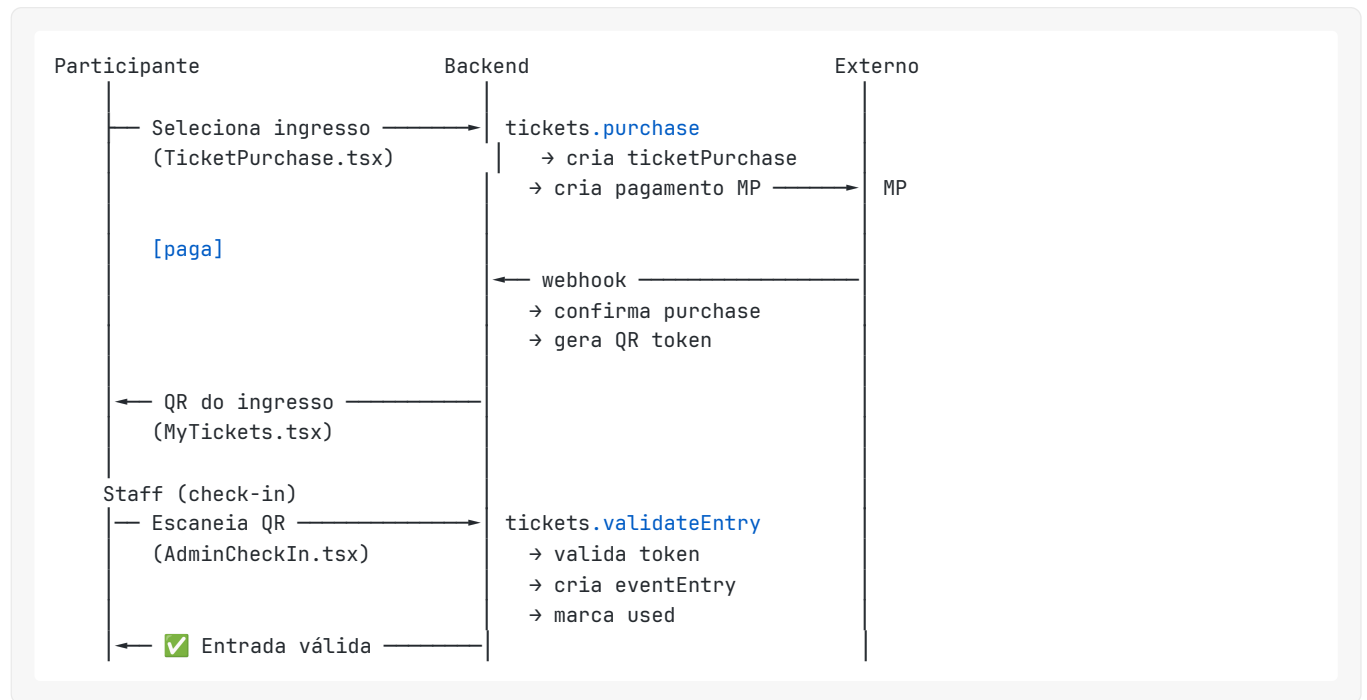
Arquivos envolvidos:

- `server/routers/orders.ts` (create, updateStatus)
- `server/mercadopago.ts` (createPixPayment)
- `server/_core/index.ts` (webhook handler)
- `server/dispatch.ts` (Smart Routing)
- `server/pushNotification.ts` (push)

- `client/src/pages/Menu.tsx` (UI)
- `client/src/contexts/CartContext.tsx` (carrinho)

Testes: `server/payment-flow.test.ts`, `server/mercadopago.test.ts`

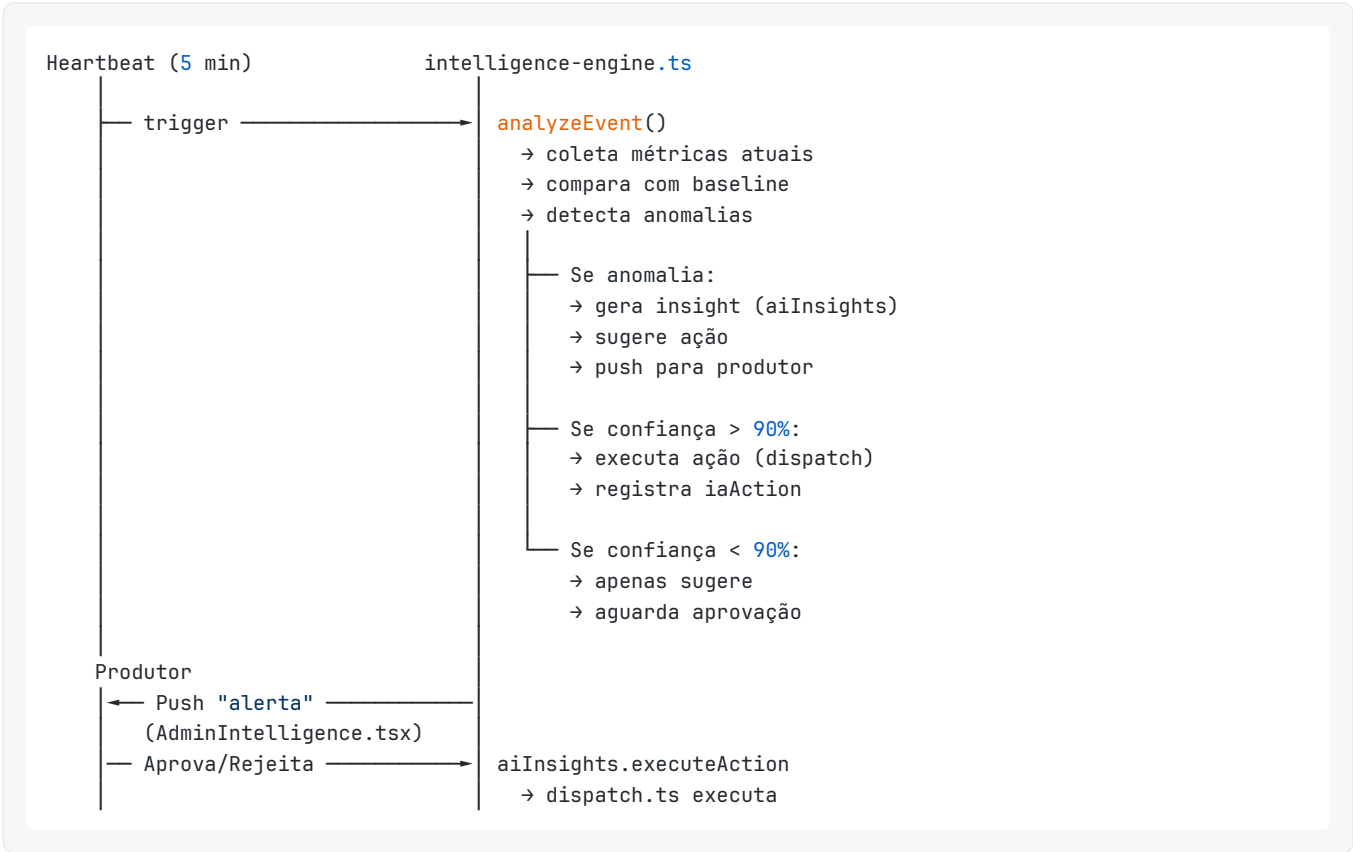
2. Ingresso → Pagamento → QR → Check-in



Arquivos: `server/routers/tickets.ts`, `client/src/pages/TicketPurchase.tsx`, `AdminCheckIn.tsx`

Testes: `server/tickets.test.ts`

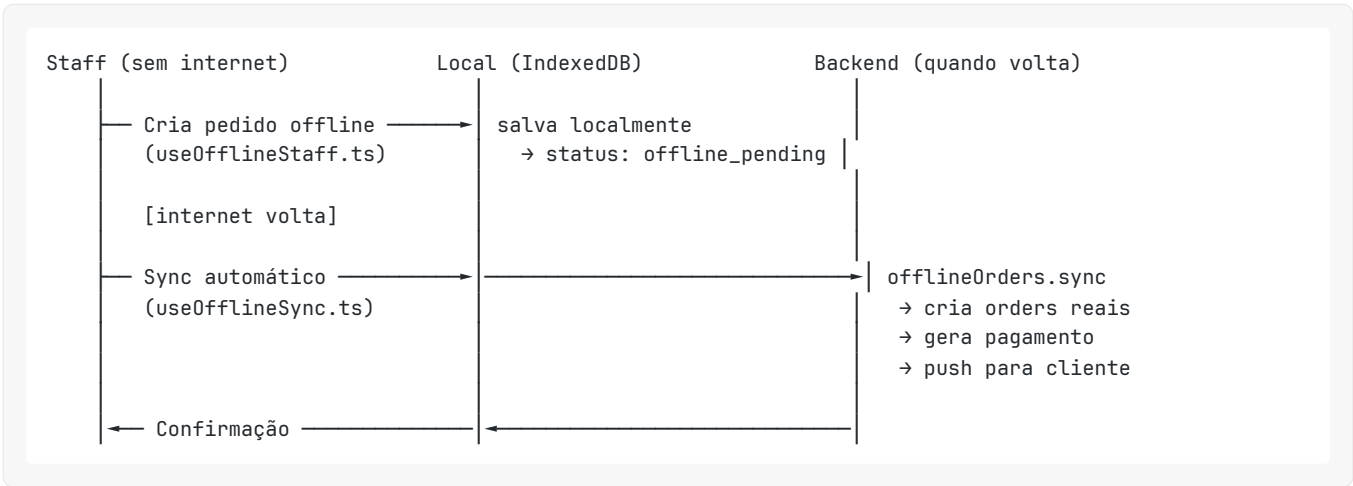
3. Syntropy → Snapshot → Baseline → Anomalia → Sugestão → Ação



Arquivos: server/intelligence-engine.ts , server/scheduled-ia-analysis.ts , server/dispatch.ts , server/routers/aiInsights.ts

Testes: server/intelligence-engine.test.ts , server/intelligence.test.ts

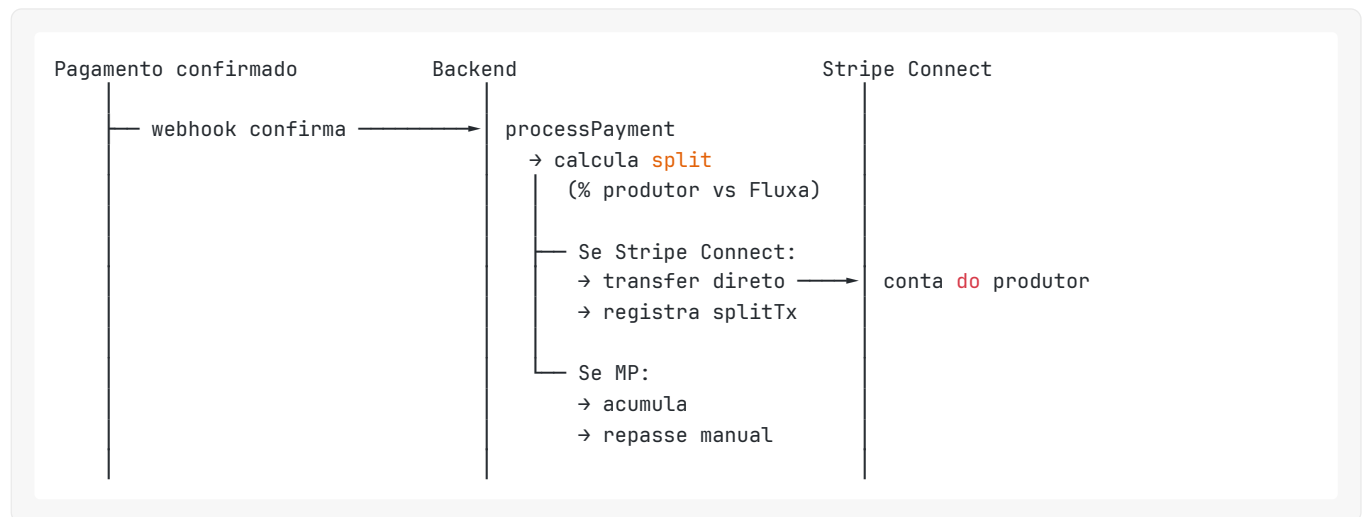
4. Pedidos Offline → Sync → Pagamento Posterior



Arquivos: server/routers/offlineOrders.ts , client/src/hooks/useOfflineStaff.ts , useOfflineSync.ts

Testes: server/offline-mode.test.ts

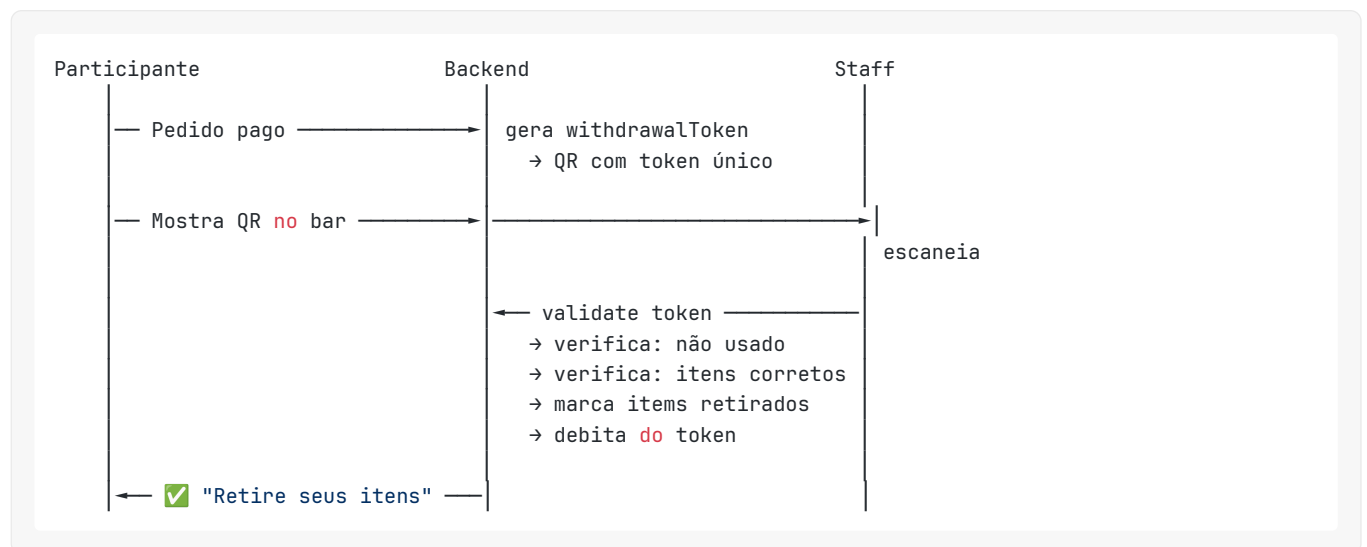
5. Split → Produtor Recebe Direto



Arquivos: `server/routers/stripeConnect.ts`, `server/routers/orders.ts`

Testes: `server/stripe-connect.test.ts`

6. Retirada de Itens (QR)



Arquivos: `server/routers/withdrawal.ts`, `client/src/pages/Withdrawal.tsx`, `StaffWithdrawal.tsx`

Testes: `server/withdrawal.test.ts`, `server/offline-withdrawal.test.ts`

Regra de Ouro

Se você vai alterar qualquer arquivo listado acima:

1. Rode `pnpm test` ANTES de começar
2. Faça a alteração
3. Rode `pnpm test` DEPOIS
4. Se quebrou algum teste → reverta
5. Se não tem teste para o que você alterou → escreva um antes

Fluxa — Guia do Desenvolvedor

Leia isto primeiro. Este é o ponto de entrada para qualquer dev (humano ou IA) que vai trabalhar no projeto.

O que é a Fluxa

A Fluxa é o **sistema operacional para eventos**. Ela controla pedidos, pagamentos, estoque, equipe e inteligência — tudo em tempo real, durante o evento.

A **Syntropy** é o cérebro da plataforma: uma inteligência que observa, decide e age autonomamente para maximizar o resultado do evento.

Domínio: <https://fluxa.bar>

Ordem de Leitura

#	Documento	Tempo	O que aprende
1	docs/FLUXA_CANON.md	5 min	O que é a Fluxa, missão, visão
2	AGENTS.md	5 min	Regras invioláveis do projeto
3	docs/02_SISTEMA/REPO_MAP.md	5 min	Onde está cada coisa no código
4	docs/02_SISTEMA/FEATURE_TO_FILE_MAP.md	3 min	Feature → arquivo
5	docs/02_SISTEMA/CRITICAL_FLOWS.md	10 min	O que não pode quebrar
6	docs/02_SISTEMA/BACKEND_ARCHITECTURE.md	10 min	Routers, procedures, webhooks
7	docs/02_SISTEMA/FRONTEND_ARCHITECTURE.md	10 min	Páginas, componentes, hooks
8	docs/02_SISTEMA/DATA_MODEL.md	10 min	Tabelas e relações
9	docs/10_SETUP_DEV/DEV_ONBOARDING.md	5 min	Como rodar e testar
10	docs/10_SETUP_DEV/CHANGE_RULES.md	5 min	Regras para alterar código

Total: ~70 minutos para entender o projeto inteiro.

Quick Start

```
pnpm install
pnpm dev
# → http://localhost:3000
```


Documentação Completa

```
docs/
├── FLUXA_CANON.md           ← Verdade oficial
├── SYSTEM_MAP.md           ← Mapa visual da plataforma
├── VANTAGEM_COMPETITIVA.md ← Por que a Fluxa ganha
├── NETWORK_EFFECT.md       ← Efeito de rede
├── FASE1_VISAO_VS_CODIGO.md ← Gap analysis (visão vs implementação)
├── 000_VISAO/              ← Visão do produto
│   ├── A_EMPRESA.md
│   ├── B_PRODUTO.md
│   ├── C_FILOSOFIA.md
│   ├── D_PRINCIPIOS.md
│   ├── E_O_QUE_NAO_E.md
│   └── F_JORNADA_PRODUTOR.md
├── 02_SISTEMA/              ← Arquitetura técnica
│   ├── REPO_MAP.md
│   ├── FEATURE_TO_FILE_MAP.md
│   ├── BACKEND_ARCHITECTURE.md
│   ├── FRONTEND_ARCHITECTURE.md
│   ├── DATA_MODEL.md
│   └── CRITICAL_FLOWS.md
├── 04_SYNTROPY/             ← Inteligência artificial
│   ├── README.md
│   ├── DECISION_ENGINE.md
│   ├── KNOWLEDGE_GRAPH.md
│   ├── INFERENCE_ENGINE.md
│   ├── LEARNING_ENGINE.md
│   ├── MEMORY_ENGINE.md
│   ├── CONTEXT_ENGINE.md
│   ├── CONFIDENCE_ENGINE.md
│   ├── GOAL_ENGINE.md
│   ├── ETHICS_ENGINE.md
│   ├── SIMULATION_ENGINE.md
│   ├── EVOLUTION.md
│   ├── VISION.md
│   ├── SMART_ROUTING.md
│   ├── FORECAST.md
│   └── PROMOCOES.md
├── 10_SETUP_DEV/            ← Setup e regras
│   ├── DEV_ONBOARDING.md
│   └── CHANGE_RULES.md
```

Regras Rápidas

1. Rode `pnpm test` antes e depois de qualquer alteração
2. Nunca mexer em pagamentos sem teste
3. Nunca mexer em estoque sem teste
4. Nunca mexer na Syntropy sem atualizar docs
5. Imagens → S3 (nunca no repo)
6. Secrets → Manus (nunca no código)

Contato

- **Produto:** Nanda (Maria Fernanda Carmesini Braga)
 - **Plataforma:** Manus AI
 - **Domínio:** fluxa.bar
-

Dev Onboarding

Como rodar, testar e deployar a Fluxa.

Tempo estimado de setup: 5 minutos (se tiver acesso ao Manus).

Pré-requisitos

- Node.js 22+
 - pnpm (gerenciador de pacotes)
 - Acesso ao projeto Manus (para variáveis de ambiente)
-

Setup Local

```
# 1. Clone
git clone <repo-url> base-x
cd base-x

# 2. Instale dependências
pnpm install

# 3. Variáveis de ambiente
# As variáveis são injetadas automaticamente pelo Manus.
# Em desenvolvimento local, crie um .env com:
#   DATABASE_URL=...
#   JWT_SECRET=...
#   MP_ACCESS_TOKEN=...
#   (ver lista completa abaixo)

# 4. Rode
pnpm dev
# → Frontend: http://localhost:5173
# → Backend: http://localhost:3000
```

Variáveis de Ambiente

Obrigatórias (sistema)

Variável	Descrição
DATABASE_URL	Connection string MySQL/TiDB
JWT_SECRET	Secret para cookies de sessão
VITE_APP_ID	ID do app Manus OAuth
OAuth_SERVER_URL	URL do servidor OAuth
VITE_OAuth_PORTAL_URL	URL do portal de login
OWNER_OPEN_ID	OpenID do owner
BUILT_IN_FORGE_API_URL	URL da API interna Manus
BUILT_IN_FORGE_API_KEY	Key da API interna

Mercado Pago (PRODUÇÃO — cuidado!)

Variável	Descrição
MP_ACCESS_TOKEN	Token de acesso MP (produção real)
MP_WEBHOOK_SECRET	Secret do webhook MP
VITE_MP_PUBLIC_KEY	Chave pública MP (frontend)

Stripe (sandbox)

Variável	Descrição
STRIPE_SECRET_KEY	Secret key Stripe (test)
STRIPE_WEBHOOK_SECRET	Secret do webhook Stripe
VITE_STRIPE_PUBLISHABLE_KEY	Publishable key (frontend)

Push Notifications

Variável	Descrição
VAPID_PUBLIC_KEY	Chave pública VAPID
VAPID_PRIVATE_KEY	Chave privada VAPID
VITE_VAPID_PUBLIC_KEY	Chave pública (frontend)

Outros

Variável	Descrição
SAKANA_API_KEY	API de IA (Sakana)
VITE_APP_TITLE	Título do app
VITE_APP_LOGO	URL do logo

Serviços Externos

Serviço	Ambiente	Cuidado
Mercado Pago	PRODUÇÃO	Pagamentos reais. Testar com R\$ 1,00
Stripe	Sandbox	Cartão de teste: 4242 4242 4242 4242
TiDB	Produção	Banco real. Cuidado com migrations
Manus OAuth	Produção	Login real
Push (VAPID)	Produção	Notificações reais

Como Testar

Testes unitários

```
pnpm test # Roda todos os testes
pnpm test -- --run # Roda sem watch mode
```

Testes E2E

```
pnpm playwright test
```

Teste manual de pagamento

Pix (dinheiro real):

1. Acesse um evento → cardápio
2. Faça pedido → escolha Pix
3. Pague R\$ 1,00 com seu banco
4. Observe webhook confirmar

Cartão (Stripe sandbox):

1. Mesmo fluxo → escolha cartão
2. Use 4242 4242 4242 4242 , validade futura, CVC 123
3. Não cobra nada real

Como Deployar

O deploy é automático via Manus:

1. Faça as alterações
2. Rode `pnpm test` — tudo verde
3. Salve checkpoint (`webdev_save_checkpoint`)
4. Clique “Publish” na UI do Manus

Domínio: <https://fluxa.bar>

Banco de Dados

Ver schema atual

```
# 0 schema está em drizzle/schema.ts
```

Criar migração

```
pnpm drizzle-kit generate
# → Gera SQL em drizzle/XXXX_*.sql
# → Aplique via webdev_execute_sql
```

Regras

- **Nunca** editar SQL gerado manualmente
 - **Nunca** rodar DROP TABLE sem backup
 - **Sempre** testar migração em dev antes de prod
 - Schema e banco devem estar sempre em sync
-

Estrutura de Branches

- `main` — produção (auto-deploy)
 - Não há branches de feature — desenvolvimento direto no main via Manus
-

Cuidados Especiais

Área	Cuidado
Mercado Pago	É PRODUÇÃO. Qualquer pagamento é real.
Webhooks	Se alterar URL ou handler, pagamentos param de confirmar
Estoque	Débito acontece no momento do pedido, não do pagamento
Push	Precisa de HTTPS para funcionar
Offline	Testar com DevTools → Network → Offline
Smart Routing	Se quebrar, pedidos não chegam no bar certo

Ordem de Leitura Recomendada

Para um dev novo entender o projeto:

1. docs/FLUXA_CANON.md — O que é a Fluxa (5 min)
2. docs/02_SISTEMA/REPO_MAP.md — Onde está cada coisa (5 min)
3. docs/02_SISTEMA/FEATURE_TO_FILE_MAP.md — Feature → arquivo (3 min)
4. docs/02_SISTEMA/CRITICAL_FLOWS.md — O que não pode quebrar (10 min)
5. docs/02_SISTEMA/BACKEND_ARCHITECTURE.md — Como o backend funciona (10 min)
6. AGENTS.md — Regras para trabalhar no projeto (5 min)

Change Rules

Regras para qualquer alteração no código da Fluxa.

Quebre essas regras e o evento pode parar.

Regras Invioláveis

1. Nunca mexer em pagamentos sem teste

Arquivos protegidos:

- server/routers/orders.ts
- server/routers/stripe.ts
- server/mercadopago.ts
- server/_core/index.ts (webhooks)

Antes de alterar:

```
pnpm test -- payment-flow
pnpm test -- mercadopago
pnpm test -- stripe
```

Se quebrar qualquer teste → reverta imediatamente.

2. Nunca mexer em estoque sem teste

Arquivos protegidos:

- Lógica de débito em `server/routers/orders.ts`
- `server/routers/ingredients.ts`
- `server/routers/products.ts` (stock-related)

Risco: Débito duplo, estoque negativo, produto vendido sem ter.

3. Nunca mexer na Syntropy sem atualizar documentação

Arquivos:

- `server/intelligence-engine.ts`
- `server/dispatch.ts`
- `server/scheduled-ia-analysis.ts`

Se alterar lógica de decisão:

- Atualizar `docs/04_SYNTROPY/DECISION_ENGINE.md`
 - Atualizar `docs/02_SISTEMA/CRITICAL_FLOWS.md` (fluxo 3)
-

4. Toda feature nova precisa atualizar o mapa

Ao criar nova funcionalidade:

1. Adicionar em `docs/02_SISTEMA/FEATURE_TO_FILE_MAP.md`
 2. Se criar novo router → adicionar em `BACKEND_ARCHITECTURE.md`
 3. Se criar nova página → adicionar em `FRONTEND_ARCHITECTURE.md`
 4. Se criar nova tabela → adicionar em `DATA_MODEL.md`
-

5. Toda mudança crítica precisa ter rollback

Definição de “mudança crítica”:

- Altera schema do banco
- Altera fluxo de pagamento
- Altera webhook
- Altera autenticação
- Altera Smart Routing

Procedimento:

1. Salvar checkpoint ANTES
 2. Fazer a alteração
 3. Testar
 4. Se falhar → rollback para o checkpoint
-

6. Nunca alterar `server/_core/` sem necessidade extrema

Esses arquivos são framework. Se alterar:

- OAuth pode quebrar
- Autenticação pode quebrar
- Webhooks podem parar
- Deploy pode falhar

Exceção: `index.ts` para adicionar middleware de segurança.

7. Nunca commitar secrets ou .env

- Variáveis de ambiente são gerenciadas pelo Manus
- Nunca hardcodar tokens, keys ou secrets no código
- Se precisar de nova variável → usar `webdev_request_secrets`

8. Imagens nunca no repositório

- Usar `manus-upload-file --webdev` para upload
- Usar URL retornada no código
- Armazenar original em `/home/ubuntu/webdev-static-assets/`

Checklist Pré-Deploy

```
[ ] pnpm test -- --run (todos passando)
[ ] Nenhum console.log de debug
[ ] Nenhum TODO crítico aberto
[ ] Feature documentada no mapa
[ ] Se mexeu em pagamento → testou manualmente
[ ] Se mexeu em schema → migração aplicada
[ ] Checkpoint salvo
```

Severidade de Áreas

Área	Severidade	Se quebrar...
Pagamentos (orders, webhooks)	● CRÍTICA	Ninguém compra
Smart Routing (dispatch)	● CRÍTICA	Pedidos não chegam no bar
Estoque	● ALTA	Vende sem ter
KDS (stationTokens)	● ALTA	Bar não vê pedidos
Push notifications	● MÉDIA	Cliente não sabe que pedido ficou pronto
Syntropy (intelligence)	● MÉDIA	Produtor perde insights
Admin pages	● BAIXA	Produtor não vê dados (mas evento continua)
Landing page	● BAIXA	Marketing afetado, operação ok

Comunicação

Ao fazer alteração significativa:

1. Descrever o que mudou no commit/checkpoint
2. Se afeta outro dev → avisar
3. Se afeta produção → testar em staging primeiro (ou com valor mínimo)